Quantitative Research

Part 5: Classification

Prof. Dr. Christoph Weisser

About the Lecturer



Prof. Dr. rer. pol. Christoph Weisser

MLitt (St Andrews), MSc (Oxford)

Professor of Mathematics, specialising in Business Data Science

- Former Technical Lead for Artificial Intelligence and supervisor of industrial doctoral projects at BASF
- Research: Data Science, Machine Learning and Artificial Intelligence
- Focus: Application of AI in business and industry

Course materials:
github.com/ChrisW09/Quantitative-Research-Methods

The lecture at a glance

	Part	Content
	1 – Descriptive Statistics	Basic concepts, levels of measurement, frequencies, measures of central tendency and dispersion
	2 – Graphical Representations	Empirical CDF, histogram, boxplot
	3 – Statistical Inference	Normal distribution, CLT, confidence intervals, sample size, hypothesis tests, p-values, one-sided tests, types of error
	4 – Regression	Correlation, least squares, goodness of fit, inference for regression, multiple regression (appendix: residual analysis)
▶	**5 – Classification**	**Conditional probability, logistic regression, classification rules, confusion matrix, ROC/AUC, Bayes classifiers**
	Python course	Separate course: Python basics, pandas, plots, machine learning (Colab notebooks)

All analyses in **Python** (NumPy, pandas, SciPy, statsmodels, matplotlib); every deck comes with a Python script and a Jupyter notebook – also in Google Colab.

Contents – Part 5

1. Classification Problems
2. Logistic Regression
3. Multiple Logistic Regression
4. From Probabilities to Decisions
5. Evaluating a Classifier
6. Summary

▶ **Try it yourself:** notebook, all examples and exercises of this part – open in Colab

The common thread of this part

One question runs through every section

The variable Y is no longer metric but **categorical** (yes/no). How do we estimate the **conditional probability** of "yes" given the data x – and how do we turn it into a good **decision**?

Three steps:

- **Model** $P(Y = 1 \mid x)$ – from a two-way table to logistic regression, a linear model for the log-odds (Part 4 on a new scale).
- **Decide** – a threshold turns a probability into a class.
- **Evaluate** – confusion matrix, precision and base rate, ROC curve.

One running story:

Credit card default

A bank observes $n = 10\,000$ customers: did they **default** on their credit card debt? Explanatory variables: the card **balance**, the **income** and whether the customer is a **student**.

Learning objectives – Part 5

After this part you will be able to ... (and to carry out every step in Python)

- read **joint, marginal and conditional** relative frequencies from a two-way table and check **independence**,
- explain why **linear regression fails** for a 0/1 variable,
- convert between **probability, odds and log-odds**,
- interpret the coefficients of a **logistic regression** (sign, odds ratio, probability),
- carry out z-**tests** and compute **confidence intervals** for coefficients and odds ratios,
- turn probabilities into decisions with a **threshold**,
- compute **accuracy, sensitivity, specificity and precision** and explain the role of the **base rate**,
- read a **ROC curve** and interpret the **AUC**,
- evaluate a classifier honestly on **test data**.

Key takeaway

Classification = a conditional probability *plus* a decision rule – both evaluated honestly.

→ *Appendix: total probability and Bayes' theorem, naive Bayes, LDA*

List of symbols (1/2)

Symbol	Meaning	Symbol	Meaning
$P(A \mid B)$	Probability of A given B	$A \cap B$, $A \cup B$	"A and B", "A or B"
$n(A)$	Number of observations in A	$\bar{A}$	"not A" (complement)
$Y \in \{0, 1\}$	Binary response ($1 =$ "yes")	$X_1, \ldots, X_p$	Explanatory variables
$p(x)$	$P(Y = 1 \mid X = x)$	$\hat{p}(x)$	Estimated probability
$\frac{p}{1-p}$	Odds	$\log \frac{p}{1-p}$	Log-odds (logit)
$\eta = \beta_0 + \beta_1 x$	Linear predictor (log-odds)	e^{β_j}	Odds ratio
$L(\beta)$	Likelihood	$\hat{\beta}_j$	Maximum likelihood estimate
Z	z-statistic $\hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j)$	$z_{1-\alpha/2}$	Quantile of $N(0, 1)$

List of symbols (2/2)

Symbol	Meaning	Symbol	Meaning
t	Threshold of the classification rule	$\hat{y}$	Predicted class
TP, TN	True positives / negatives	FP, FN	False positives / negatives
Sens.	Sensitivity $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$	Spec.	Specificity $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$
Prec.	Precision $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$	π	Base rate $P(Y=1)$
ROC	Receiver operating characteristic	AUC	Area under the ROC curve
π_k	Prior probability of class k	$f_k(x)$	Density of X in class k

As in Parts 3 and 4: H_0, H_1, α, p-value, confidence level $1-\alpha$, standard error SE.

Overview

1 Classification Problems
- What is classification?
- Two-way tables and conditional probability
- Why not linear regression?

2 Logistic Regression

3 Multiple Logistic Regression

4 From Probabilities to Decisions

5 Evaluating a Classifier

6 Summary

From regression to classification

Regression (Part 4)

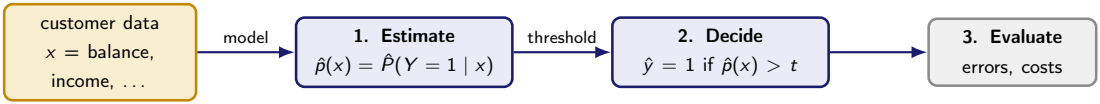
The variable to be explained Y is **metric**: sales quantity, shoe size, card balance.

Classification (Part 5)

Y is **categorical** – in this part *binary*: yes/no, coded $Y = 1$ ("yes") and $Y = 0$ ("no").

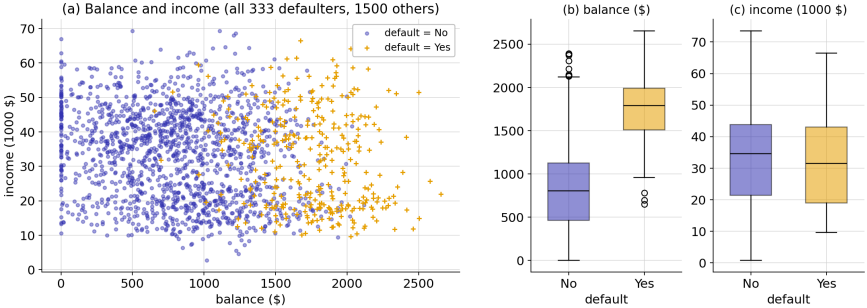
Question	Y	**Explanatory variables**
Will the customer default?	default yes/no	balance, income, student
Is this e-mail spam?	spam yes/no	words, links, sender
Will the customer churn?	churn yes/no	usage, contract, complaints
Is the patient ill?	diagnosis yes/no	test results, age

Three steps in every classification problem



- The **probability** is a statistical question – answered by a model such as logistic regression (Sections 2–3).
- The **decision** is also a business question – which error is more expensive? (Sections 4–5)

Our story: the Default data



- $n = 10\,000$ customers, of which only 333 (3.33 %) defaulted.
- Defaulters have clearly **higher balances**; income hardly differs between the groups.

Source: Default data from James, Witten, Hastie, Tibshirani: An Introduction to Statistical Learning, www.statlearning.com

Default by student status – a two-way table

Guiding question

Do students default more often than non-students? Before building a model, we simply **count**.

Counts $n(\cdot)$

	no default	default	Total
non-student	6850	206	7056
student	2817	127	2944
Total	9667	333	10 000

Relative frequencies $n(\cdot)/n$

	no default	default	Total
non-student	0.6850	0.0206	**0.7056**
student	0.2817	**0.0127**	**0.2944**
Total	**0.9667**	**0.0333**	1

- **Joint** relative frequency (inner cells): 1.27 % of all customers are students *and* defaulted.
- **Marginal** relative frequencies (**bold** totals): 29.44 % are students, 3.33 % defaulted – each variable on its own.

Conditional relative frequencies – change the denominator

"Among the students, what share defaulted?"

Look only at the row "student" and divide by *its* total:

$$\frac{n(\text{student} \cap \text{default})}{n(\text{student})} = \frac{127}{2944} = 0.0431 \qquad \left(= \frac{0.0127}{0.2944} = \frac{\text{joint}}{\text{marginal}}\right)$$

	no default	default	Total
non-student	0.9708	**0.0292**	1
student	0.9569	**0.0431**	1
all customers	0.9667	0.0333	1

each row divided by its row total

- Students default **more often**: 4.31 % vs. 2.92 % (overall 3.33 %).
- The other direction is a *different* question: $\frac{127}{333} = 0.381$ – 38 % of the *defaulters* are students (divide by the column total).

In words: Conditioning = restricting attention to one row (or column) and dividing by its total.

Conditional probability – definition and two rules

Conditional probability of A given B

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \qquad (P(B) > 0)$$

In words: the share of the cases in B that also belong to A (a row or column share of the table).

Multiplication rule ("A and B")

$$P(A \cap B) = P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$$

Addition rule ("A or B")

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Default data (A = default, B = student)

$P(A \mid B) = 0.0127/0.2944 = 0.0431$; and back: $P(A \cap B) = 0.0431 \cdot 0.2944 = 0.0127$.
$P(A \cup B) = 0.0333 + 0.2944 - 0.0127 = 0.3150$: 31.5 % are students *or* defaulters (or both).

Relative frequencies in the data *estimate* these probabilities, e.g. $\hat{P}(\text{default} \mid \text{student}) = 0.0431$.

Independence

Definition

Two events A and B are **independent** if

$$P(A \cap B) = P(A)\,P(B) \qquad \Longleftrightarrow \qquad P(A \mid B) = P(A) \quad (P(B) > 0).$$

In words: knowing that B occurred does not change the probability of A.

Default data: are default and student status independent?

$P(\text{default})\,P(\text{student}) = 0.0333 \cdot 0.2944 = 0.0098 \neq 0.0127 = P(\text{default} \cap \text{student})$
$\Rightarrow$ **not independent**; equivalently $P(\text{default} \mid \text{student}) = 0.0431 \neq 0.0333$.
Independence would mean $10\,000 \cdot 0.0098 = 98$ student defaulters – we observe 127.

Key takeaway

If Y were independent of X, knowing X would not help to predict Y: classification lives on the **dependence** of Y on X.

Classification = estimating $P(Y = 1 \mid x)$

- For a **categorical** x (student yes/no) the two-way table gives $\hat{P}(\text{default} \mid x)$ directly.
- **Balance** is metric: almost every customer has a different value. A first attempt – group the balances into classes:

balance class ($)	$[0, 500)$	$[500, 1000)$	$[1000, 1500)$	$[1500, 2000)$	≥ 2000
customers	2613	3755	2721	805	106
defaulters	0	6	69	177	81
default rate $\hat{P}(\text{default} \mid \text{class})$	0.000	0.002	0.025	0.220	0.764

In words: The conditional default rate first hardly moves, then rises steeply – an **S-shape**. But the classes are crude: a customer with 1999 $ is treated like one with 1500 $.

Key takeaway

Classification means estimating the **conditional probability** $p(x) = P(Y = 1 \mid X = x)$ as a smooth function of x – this needs a **model**.

Two-way tables with Python

```python
import pandas as pd
df = pd.read_csv("../data/Default.csv")                    # n = 10 000 customers
tab = pd.crosstab(df.student, df.default, margins=True)    # counts + totals
print(tab)
print(pd.crosstab(df.student, df.default, normalize="index").round(4))
```

```
default      No  Yes    All
student
No         6850  206   7056
Yes        2817  127   2944
All        9667  333  10000
default       No     Yes
student
No        0.9708  0.0292
Yes       0.9569  0.0431
```

Option	Result
`margins=True`	adds row and column totals
`normalize="all"`	joint relative frequencies
`normalize="index"`	each **row** sums to 1
`normalize="columns"`	each **column** sums to 1

In words: `"index"` conditions on the row variable (student status), `"columns"` on the column variable.

▶ **Try it yourself:** notebook, section 1.2 "Two-way tables and conditional probability" – open in Colab

Exercise – newsletter and purchases

Problem statement

An online shop records for $n = 60$ randomly chosen customers whether they subscribe to the newsletter and whether they bought something last month.

newsletter \ purchase	yes	no	Total
yes	15	5	20
no	15	25	40
Total	30	30	60

1. Compute the marginal relative frequencies and the joint one of (yes, yes). Are newsletter and purchase independent? How many subscribing buyers would independence predict?
2. Compute the purchase rates of subscribers and of non-subscribers and compare them with the overall rate.
3. Compute the share of subscribers among the buyers. Why does it differ from the purchase rate of the subscribers?

Solution – newsletter and purchases

(1) Marginal: $\frac{20}{60} = 0.333$ subscribe, $\frac{40}{60} = 0.667$ do not; $\frac{30}{60} = 0.5$ buy. Joint: $\frac{15}{60} = 0.25$.

$$\text{independence would require } 0.333 \cdot 0.5 = 0.167 \neq 0.25 \Rightarrow \textbf{not independent}.$$

Under independence we would expect $60 \cdot 0.167 = 10$ subscribers who buy – we observe 15.

(2) Condition on the row (divide by the row total):

$$P(\text{purchase} \mid \text{subscriber}) = \frac{15}{20} = \mathbf{0.75}, \qquad P(\text{purchase} \mid \text{non-subscriber}) = \frac{15}{40} = \mathbf{0.375}$$

Subscribers buy twice as often; under independence both rates would equal the overall rate 0.5.

(3) Condition on the column "purchase yes": $P(\text{subscriber} \mid \text{purchase}) = \frac{15}{30} = \mathbf{0.5}$ – half of the buyers subscribe. Same cell (15), different denominator: 20 subscribers in (2), 30 buyers in (3).

Common mistakes

Confusing $P(A \mid B)$ with $P(B \mid A)$ – and reading the association as an effect: loyal customers may simply be more likely to subscribe.

Conditional probability and independence with Python

Default data: conditional probability, the two rules, independence:

```
d = df.default == "Yes"                          # True for the defaulters
s = df.student == "Yes"                          # True for the students
p_d, p_s, p_ds = d.mean(), s.mean(), (d & s).mean()   # 0.0333  0.2944  0.0127
print(p_ds / p_s)                                # P(default | student) = 0.0431
print(p_ds / p_s * p_s)                          # multiplication rule: back to 0.0127
print(p_d + p_s - p_ds)                          # addition rule: P(default or student) = 0.3150
print(p_d * p_s)                                 # 0.0098 != 0.0127 -> not independent
```

Default rate by balance class (slide "Classification = estimating $P(Y = 1 \mid x)$"):

```
import numpy as np
bal_class = pd.cut(df.balance, [0, 500, 1000, 1500, 2000, np.inf], right=False)
print(pd.crosstab(bal_class, df.default, normalize="index").round(3))
# column "Yes": 0.000  0.002  0.025  0.220  0.764 -- the S-shape
```

In words: the mean of a True/False column is the share of `True` values; `d & s` means "A and B".

▶ **Try it yourself:** notebook, section 1.2 "Two-way tables and conditional probability" – open in Colab

Exercise – streaming and gaming (exam style)

Problem statement

Among the users of a smartphone model, 60 % use the streaming app (S), 50 % play games (G) and 30 % do both.

1. Compute $P(G \mid S)$ and interpret it.
2. Compute $P(S \cup G)$. What share of the users does neither?
3. Check whether S and G are independent.
4. Fill in the two-way table of probabilities for S/not S and G/not G.

Solution – streaming and gaming (exam style)

(1) $P(G \mid S) = \frac{P(G \cap S)}{P(S)} = \frac{0.30}{0.60} = \mathbf{0.50}$: half of the streaming users also play games.

(2) $P(S \cup G) = 0.60 + 0.50 - 0.30 = \mathbf{0.80}$; neither: $1 - 0.80 = \mathbf{0.20}$. (Without -0.30 the users who do both are counted twice: $1.10 > 1$.)

(3) $P(S)\,P(G) = 0.60 \cdot 0.50 = 0.30 = P(S \cap G)$ $\Rightarrow$ **independent**. Equivalently $P(G \mid S) = 0.50 = P(G)$: knowing that someone streams does not change the probability that they play.

(4) Enter the given values, fill the rest by subtraction:

	G	not G	Total
S	**0.30**	0.30	**0.60**
not S	0.20	0.20	0.40
Total	**0.50**	0.50	1

given values in bold

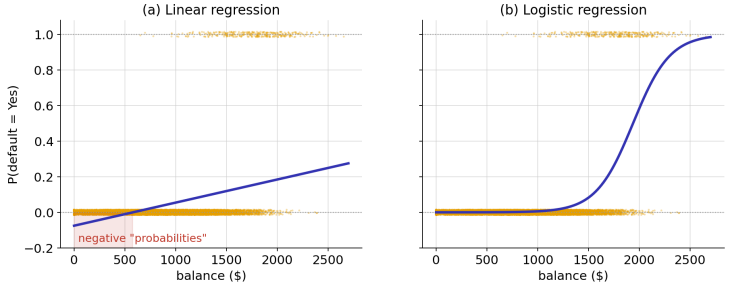
Common mistake

Independent $\neq$ disjoint: disjoint means $P(S \cap G) = 0$ – here 30 % do both.

Why not simply use linear regression?

Tempting idea

Code $Y = 1$ for default and $Y = 0$ otherwise and run the regression of Part 4: $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1$ balance.



In words: The straight line predicts *negative* default "probabilities" for 3123 customers (all balances below 579 $); the S-curve stays between 0 and 1.

Three problems of linear regression with a 0/1 variable

Problems

1. **Predictions outside** $[0, 1]$ – they cannot be read as probabilities.
2. **Heteroscedasticity by construction** – $\text{Var}(Y \mid x) = p(x)\,(1 - p(x))$ depends on x, so the standard errors and tests of Part 4 are not valid.
3. **No natural extension to more than two categories** – codes $1, 2, 3$ would impose an order and equal distances that the categories do not have.

Key takeaway

We need a model whose output is always a probability in $(0, 1)$ – the **logistic regression**.

Linear regression on a 0/1 variable with Python

```
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
df["y"] = (df.default == "Yes").astype(int)      # 1 = default, 0 = no
lpm = smf.ols("y ~ balance", data=df).fit()      # the regression of Part 4
print(lpm.params)                                # -0.0752, 0.000130
print((lpm.fittedvalues < 0).sum())              # 3123 negative "probabilities"
print(df.balance[lpm.fittedvalues < 0].max())    # all balances below 579 $
plt.scatter(df.balance, df.y, s=3)               # the 0/1 data
plt.plot(df.balance, lpm.fittedvalues, "r")      # fitted straight line
```

```
Intercept   -0.075192
balance      0.000130
dtype: float64
3123
578.836277403294
```

- $\hat{y} = -0.0752 + 0.000130 \cdot \text{balance} < 0$ for every balance below $0.0752/0.000130 \approx 579\,\$$.
- Python fits the line without any warning – checking the model is our job.

▶ **Try it yourself:** notebook, section 1.3 "Why not linear regression?" – open in Colab

Exercise – why not linear regression?

Problem statement

A colleague wants to predict the emergency diagnosis of a patient. She codes the three *unordered* categories as $Y = 1$ (stroke), $Y = 2$ (overdose), $Y = 3$ (seizure) and fits a linear regression.

1. Which implicit assumptions does the coding $1, 2, 3$ make?
2. With only two categories coded $0/1$: name two problems of the fitted values as probabilities.
3. Would relabelling the categories change the fit? What does this imply?

Solution – why not linear regression?

1. **Order and equal spacing:** the coding claims stroke $<$ overdose $<$ seizure and equal distances between them. A fitted value of 1.5 would mean "halfway between stroke and overdose" – medically meaningless.
2. **Two problems:** fitted values can lie outside $[0, 1]$ (e.g. a negative probability), and the error variance $p(1 - p)$ is not constant.
3. **Yes:** swapping the codes of stroke and seizure gives a different response and hence a different fit – although the data are the same. The result depends on an arbitrary choice, so linear regression is *not appropriate* here.

Common mistake

Hoping that more data fix the problem. The flaw lies in the model specification, not in sampling noise.

Overview

1 Classification Problems

2 Logistic Regression
- Probability, odds and log-odds
- The model
- Estimation
- Interpretation
- Inference for the coefficients

3 Multiple Logistic Regression

4 From Probabilities to Decisions

5 Evaluating a Classifier

6 Summary

Odds – another way to express a probability

Definition

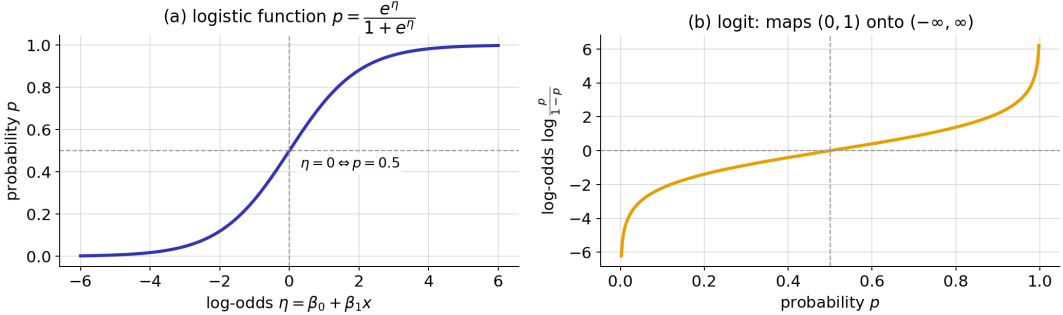
For an event with probability p:

$$\text{odds} = \frac{p}{1-p} \in (0, \infty), \qquad \text{log-odds (logit)} = \log \frac{p}{1-p} \in (-\infty, \infty).$$

Probability p	0.1	0.2	0.5	0.8	0.9
Odds $p/(1-p)$	0.111	0.25	1	4	9
Log-odds	-2.197	-1.386	0	1.386	2.197

In words: Odds of 4 mean "4 to 1": the event occurs four times as often as it does not ($p = 0.8$). Back from odds to probability: $p = \text{odds}/(1 + \text{odds})$.

Why log-odds?



- The log-odds can take **any real value** – just like the right-hand side $\beta_0 + \beta_1 x$ of a regression.
- The logistic function (a) is the inverse of the logit (b): it maps every log-odds value back into $(0, 1)$.

Odds and log-odds with Python

```
p = np.array([0.1, 0.2, 0.5, 0.8, 0.9])
odds = p / (1 - p)                          # 0.111  0.25  1  4  9
logodds = np.log(odds)                      # natural log: -2.197 ... 2.197
print(pd.DataFrame({"p": p, "odds": odds, "logodds": logodds}).round(3))
print(odds / (1 + odds))                    # back to p: 0.1 0.2 0.5 0.8 0.9
eta = np.linspace(-6, 6, 200)               # a grid of log-odds values
plt.plot(eta, np.exp(eta) / (1 + np.exp(eta)))   # logistic function, panel (a)
```

```
     p   odds  logodds
0  0.1  0.111   -2.197
1  0.2  0.250   -1.386
2  0.5  1.000    0.000
3  0.8  4.000    1.386
4  0.9  9.000    2.197
```

- The same table as on the slide "Odds" – every operation works element by element on the whole array.
- `np.log` is the **natural** logarithm (base e), `np.exp` its inverse.
- `np.exp(eta) / (1 + np.exp(eta))` turns log-odds into a probability – we need it again for $\hat{p}(x)$.

▶ **Try it yourself:** notebook, section 2.1 "Probability, odds and log-odds" – open in Colab

Exercise – probability, odds and log-odds

Problem statement

In the two-way table of Section 1, 127 of the 2944 students and 206 of the 7056 non-students defaulted.

1. Compute the probability, the odds and the log-odds of default for students and for non-students.
2. Compute the **odds ratio** (odds of the students divided by the odds of the non-students) and its logarithm. Interpret it.
3. Compare the odds ratio with the ratio of the two default probabilities. Why are the two so close here?
4. A churn model assigns a customer the log-odds -1. Compute the odds and the churn probability.

Solution – probability, odds and log-odds

(1) Odds = number of "yes" / number of "no" – straight from the table:

	probability p	odds $p/(1-p)$	log-odds log(odds)
students	$127/2944 = 0.0431$	$127/2817 = 0.0451$	-3.099
non-students	$206/7056 = 0.0292$	$206/6850 = 0.0301$	-3.504

(2) Odds ratio $= 0.0451/0.0301 = \mathbf{1.50}$, $\log 1.50 = -3.099 - (-3.504) = \mathbf{0.405}$: the odds of default of students are 1.5 times (i.e. 50 % higher than) those of non-students.

(3) Probability ratio $0.0431/0.0292 = 1.48 \approx 1.50$: for *rare* events $1 - p \approx 1$, so odds $\approx$ probability. For common events they differ: $p = 0.8$ vs. 0.5 gives a probability ratio of 1.6 but an odds ratio of $4/1 = 4$.

(4) odds $= e^{-1} = 0.368$, $p = \dfrac{0.368}{1 + 0.368} = \mathbf{0.269}$.

The logistic regression model

Model: linear on the log-odds scale

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + \beta_1 x \qquad \Longleftrightarrow \qquad p(x) = P(Y=1 \mid x) = \frac{e^{\beta_0+\beta_1 x}}{1+e^{\beta_0+\beta_1 x}}$$

- Both forms describe the *same* model: the left one shows the linearity, the right one guarantees $0 < p(x) < 1$.
- We write $\eta = \beta_0 + \beta_1 x$ for the **linear predictor** (the log-odds).
- Several explanatory variables: $\eta = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$ (Section 3).

Key takeaway

Logistic regression = the regression model of Part 4, applied to the **log-odds** instead of Y itself.

Reading the model symbol by symbol

Symbol	Meaning
Y	Binary response, 0 or 1 (default no/yes)
$p(x)$	$P(Y = 1 \mid x)$, the probability of default given x
$p(x)/(1 - p(x))$	The *odds* of default
$\eta = \log[p/(1 - p)]$	The *log-odds*, linear in x
β_1	Change of the *log-odds* per unit of x
e^{β_1}	Factor by which the *odds* change per unit of x

In words: Every probability statement goes through the pipeline

$$x \xrightarrow{\text{linear}} \eta \xrightarrow{\exp} \text{odds} \xrightarrow{\text{odds}/(1+\text{odds})} p.$$

Estimation: maximum likelihood

Idea

Least squares makes no sense for 0/1 data. Instead we choose the coefficients under which the **observed outcomes are most probable**.

Likelihood

$$L(\beta_0, \beta_1) = \prod_{i:\, y_i = 1} p(x_i) \prod_{i:\, y_i = 0} \big(1 - p(x_i)\big)$$

Each defaulter contributes $p(x_i)$, each non-defaulter $1 - p(x_i)$.

- In practice we maximise the **log-likelihood** $\log L = \sum_i \big[y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i))\big]$.
- There is no closed formula – Python solves it numerically (`smf.logit(...).fit()`).

Exercise – which parameters are more plausible?

Problem statement

Four customers with balances 1000, 1500, 2000, 2500 \$; the last two defaulted ($y = 0, 0, 1, 1$). Compare two candidate parameter sets:

$$\text{(A)}\ \ \beta_0 = -10.65,\ \beta_1 = 0.0055 \qquad\qquad \text{(B)}\ \ \beta_0 = -5,\ \beta_1 = 0.002$$

1. Compute $p(x_i)$ for all four customers under (A) and (B).
2. Compute the likelihood L for (A) and (B).
3. Which parameter set would maximum likelihood prefer?

Solution (1/2) – which parameters are more plausible?

Example: $\eta(2000) = -10.65 + 0.0055 \cdot 2000 = 0.35$, $p = e^{0.35}/(1 + e^{0.35}) = 0.5866$.

		(A)		(B)	
balance	y_i	$p(x_i)$	contribution	$p(x_i)$	contribution
1000	0	0.0058	0.9942	0.0474	0.9526
1500	0	0.0832	0.9168	0.1192	0.8808
2000	1	0.5866	0.5866	0.2689	0.2689
2500	1	0.9569	0.9569	0.5000	0.5000
L (product)		**0.5117**		0.1128	

Decision

Under (A) the observed pattern is 4.5 times as probable as under (B) $\Rightarrow$ maximum likelihood prefers **(A)**: high probabilities for the actual defaulters, low ones for the others.

Solution (2/2) – which parameters are more plausible?

The same calculation in Python – one function, two parameter sets:

```
x = np.array([1000, 1500, 2000, 2500]); y = np.array([0, 0, 1, 1])
def likelihood(b0, b1):
    p = np.exp(b0 + b1 * x) / (1 + np.exp(b0 + b1 * x))    # p(x_i)
    return np.prod(np.where(y == 1, p, 1 - p))             # product of contributions
print(likelihood(-10.65, 0.0055))      # (A) 0.5117
print(likelihood(-5, 0.002))           # (B) 0.1128
print(np.log(likelihood(-10.65, 0.0055)))   # log-likelihood of (A): -0.6701
```

- `np.where(y == 1, p, 1 - p)`: each defaulter contributes $p(x_i)$, each non-defaulter $1 - p(x_i)$ – the formula of the likelihood slide.
- Maximum likelihood searches *all* pairs (β_0, β_1) for the largest L (in practice: the largest $\log L$). `smf.logit` does this numerically – next slide.

▶ **Try it yourself:** notebook, section 2.3 "Estimation" – open in Colab

Logistic regression with Python

Fitting the model

```
import statsmodels.formula.api as smf
df = pd.read_csv("../data/Default.csv")
df["y"] = (df.default == "Yes").astype(int)      # 1 = default, 0 = no
g1 = smf.logit("y ~ balance", data=df).fit(disp=0)
print(g1.summary2().tables[1])
```

```
               Coef.  Std.Err.          z          P>|z|      [0.025     0.975]
Intercept -10.651331  0.361169 -29.491287  3.723665e-191 -11.359208 -9.943453
balance     0.005499  0.000220  24.952404  2.010855e-137   0.005067  0.005931
```

- `smf.logit` fits a logistic regression by maximum likelihood (y coded 0/1): $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$.
- The output looks like `smf.ols` – with z- instead of t-values (see "Inference").

▶ **Try it yourself:** notebook, section 2.3 "Estimation" – open in Colab

Interpreting the coefficients – three scales

Three ways to read $\hat{\beta}_1$

Sign: $\hat{\beta}_1 > 0 \Rightarrow$ a larger x raises the probability of $Y = 1$; $\hat{\beta}_1 < 0$ lowers it.

Log-odds: one unit more of x changes the log-odds by $\hat{\beta}_1$ – constant, but hard to imagine.

Odds: one unit more of x **multiplies the odds** by $e^{\hat{\beta}_1}$ **(odds ratio)** – constant and easy to communicate.

Not on the probability scale

The change in the *probability* is not constant: it is largest in the steep middle of the S-curve and small at the flat ends.

In words: Why the odds ratio? $\frac{\text{odds}(x+1)}{\text{odds}(x)} = \frac{e^{\beta_0+\beta_1(x+1)}}{e^{\beta_0+\beta_1 x}} = e^{\beta_1}$ – independent of x.

Worked example: default and balance

Model: $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$.

Three scales for 1000 $ more balance

- **Log-odds:** $+0.0055 \cdot 1000 = +5.5$.
- **Odds:** multiplied by $e^{5.5} \approx 245$.
- **Probability:** depends on the starting point:

balance ($)	1000	1500	2000	2500
$\hat{p}(x)$	0.006	0.083	0.587	0.957

Common mistake

"The odds multiply by 245" does *not* mean the probability is 245 times as large – probabilities can never exceed 1.

Odds ratios and predicted probabilities with Python

```
print(np.exp(g1.params["balance"]))          # odds ratio per 1 $:     1.0055
print(np.exp(1000 * g1.params["balance"]))   # odds ratio per 1000 $:  244.4
new = pd.DataFrame({"balance": [1000, 1500, 2000, 2500]})
eta = g1.predict(new, which="linear")        # log-odds: -5.152 -2.403 0.347 3.096
print(np.exp(eta) / (1 + np.exp(eta)))       # by hand:  0.0058 0.0829 0.5858 0.9567
print(g1.predict(new))                       # the same probabilities in one step
```

- `new` needs a column with the **same name** as in the formula (`balance`); one row per customer.
- `which="linear"` returns the log-odds $\hat{\eta}$; without it, `predict` returns $\hat{p}(x)$ – the pipeline $x \to \hat{\eta} \to \hat{p}$.
- Python uses the unrounded coefficients: odds ratio 244.4 instead of $e^{5.5} \approx 245$; the probabilities agree with the table of the worked example.

▶ **Try it yourself:** notebook, section 2.4 "Interpretation" – open in Colab

Exercise – computing $\hat{p}(x)$ from the coefficients

Problem statement

The logistic regression of default on balance gives $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$. For a customer with a balance of 1500 \$:

1. Compute the log-odds $\hat{\eta}$.
2. Compute the probability of default $\hat{p}$.
3. Compute the odds and relate them to part (1).
4. At which balance does the model give $\hat{p} = 0.5$?

Solution – computing $\hat{p}(x)$ from the coefficients

(1)–(3) Log-odds, probability, odds

$$\hat{\eta} = -10.65 + 0.0055 \cdot 1500 = -2.40, \qquad \hat{p} = \frac{e^{-2.40}}{1 + e^{-2.40}} = \frac{0.0907}{1.0907} = \mathbf{0.083},$$

$$\text{odds} = \frac{0.083}{0.917} = 0.091 = e^{-2.40} \ \checkmark \quad \text{(about one default per eleven non-defaults)}$$

(4) Balance with $\hat{p} = 0.5$

$\hat{p} = 0.5 \Leftrightarrow \text{odds} = 1 \Leftrightarrow \hat{\eta} = 0$:
$0 = -10.65 + 0.0055 \cdot \text{balance} \Rightarrow \text{balance} = 10.65/0.0055 \approx \mathbf{1936}\,\$$.

Common mistake

Reading $\hat{\eta} = -2.40$ as a probability – log-odds must first be passed through the logistic function.

Inference – the same recipe once more

z-test for a coefficient (Wald test)

$$H_0 : \beta_j = 0 \quad \text{vs.} \quad H_1 : \beta_j \neq 0, \qquad Z = \frac{\hat{\beta}_j}{\mathrm{SE}(\hat{\beta}_j)} \;\approx\; N(0,1) \;\text{ under } H_0$$

Reject if $|Z| > z_{1-\alpha/2}$ (1.96 for $\alpha = 0.05$); $p = 2\,\Phi(-|Z|)$.

Confidence intervals

$$\beta_j : \quad \hat{\beta}_j \pm z_{1-\alpha/2}\,\mathrm{SE}(\hat{\beta}_j), \qquad \text{odds ratio } e^{\beta_j} : \quad \left[e^{\text{lower}},\, e^{\text{upper}}\right]$$

- Why z and not t? Maximum likelihood estimators are approximately normal for large n – as for the known-variance case of Part 3.
- The CI for the odds ratio is obtained by **exponentiating the bounds** – it is not symmetric around $e^{\hat{\beta}_j}$.

Inference – balance and student status

Balance ($\hat{\beta}_1 = 0.005499$, SE $= 0.000220$)

$$Z = \frac{0.005499}{0.000220} = 24.95, \qquad p < 0.0001, \qquad \text{CI}_{0.95}(\beta_1) = [\,0.005067\,;\,0.005931\,]$$

Per 100 \$: odds ratio $e^{0.5499} = 1.733$, CI $[e^{0.5067};\, e^{0.5931}] = [\,1.660\,;\,1.810\,]$ – every 100 \$ of balance raises the odds of default by 66 to 81 %.

Student ($\hat{\beta} = 0.4049$, SE $= 0.1150$), `default` $\sim$ `student`

$$Z = \frac{0.4049}{0.1150} = 3.52, \qquad p = 0.0004, \qquad e^{0.4049} = 1.50, \quad \text{CI} = [\,1.20\,;\,1.88\,]$$

Students have 50 % higher odds of default – *on their own* (more on this in Section 3). With a single dummy the model reproduces the two-way table: $\hat{\beta}_0 = -3.504$ is the log-odds of the non-students and 1.50 the odds ratio (exercise on odds).

Inference with Python

Balance (z-test and CIs by hand and built in):

```
from scipy import stats
z = g1.params / g1.bse                          # balance: z = 24.95
print(2 * stats.norm.cdf(-abs(z)))              # p = 2 Phi(-|z|) = 2.0e-137 (= g1.pvalues)
print(g1.conf_int())                            # beta +- 1.96 SE: [0.005067, 0.005931]
print(np.exp(100 * g1.conf_int().loc["balance"]))   # OR per 100 $: [1.660, 1.810]
```

Student status alone:

```
g2 = smf.logit("y ~ student", data=df).fit(disp=0)
print(g2.pvalues["student[T.Yes]"])             # p = 0.0004
print(np.exp(g2.params))                        # odds ratio student: 1.50
print(np.exp(g2.conf_int()))                    # its 95 % CI: [1.20, 1.88]
```

In words: `conf_int()` gives the CI for β_j; `np.exp` of the bounds gives the CI for the odds ratio e^{β_j}. For 99 %: `conf_int(alpha=0.01)`.

▶ **Try it yourself:** notebook, section 2.5 "Inference for the coefficients" – open in Colab

Exercise – coefficients on the odds scale

Problem statement

The logistic regression of default on balance gives $\hat{\beta}_1 = 0.0055$ (per \$) with $\text{SE}(\hat{\beta}_1) = 0.00022$.

1. By how much do the *log-odds* change for 500 \$ more balance?
2. By which factor do the *odds* change for 500 \$ more balance?
3. Test $H_0 : \beta_1 = 0$ at $\alpha = 0.05$.
4. Why can we state a constant effect on the odds but not on the probability?

Solution – coefficients on the odds scale

(1) $\Delta\eta = 0.0055 \cdot 500 = \mathbf{2.75}$. **(2)** $e^{2.75} = \mathbf{15.6}$: the odds are about 16 times as large.

(3) $Z = 0.0055/0.00022 = 25.0 > 1.96 \Rightarrow$ reject H_0: balance is a highly significant predictor.

(4) The log-odds are linear in x, so a fixed Δx always shifts them by $\beta_1 \Delta x$ and multiplies the odds by $e^{\beta_1 \Delta x}$. The probability is the S-shaped logistic function of the log-odds:

start	odds before	odds after ($\times 15.6$)	probability after
$p = 0.10$	0.111	1.74	0.63 (+53 points)
$p = 0.90$	9	141	0.993 (+9 points)

Overview

1 Classification Problems

2 Logistic Regression

3 Multiple Logistic Regression
- The model
- Confounding: the student paradox
- Exercises

4 From Probabilities to Decisions

5 Evaluating a Classifier

6 Summary

Several explanatory variables

Model

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Everything from Part 4 carries over:

- β_j is the change of the log-odds per unit of x_j **holding all other variables fixed**; e^{β_j} is the corresponding odds ratio.
- Qualitative variables enter as **dummies** (e.g. `student[T.Yes]` $= 1$ for students).
- Simple and multiple coefficients can differ completely – **confounding**.

Multiple logistic regression with Python

```
g3 = smf.logit("y ~ balance + income + student", data=df).fit(disp=0)
print(g3.summary().tables[1])
```

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept    -10.8690      0.492    -22.079      0.000     -11.834      -9.904
student[T.Yes]  -0.6468      0.236     -2.738      0.006      -1.110      -0.184
balance       0.0057      0.000     24.737      0.000       0.005       0.006
income     3.033e-06   8.2e-06      0.370      0.712    -1.3e-05    1.91e-05
==============================================================================
```

- **Balance:** highly significant, positive.
- **Income:** not significant given balance and student ($p = 0.71$).
- **Student:** significant and now *negative*: $e^{-0.6468} = 0.524$.

▶ **Try it yourself:** notebook, section 3.1 "The model" – open in Colab

The student paradox

Alone: `default` $\sim$ `student`

$\hat{\beta}_{\text{student}} = +0.405$ ($p = 0.0004$)
Students default *more often*:
4.31 % vs. 2.92 % (two-way table, Section 1).

Together with balance and income

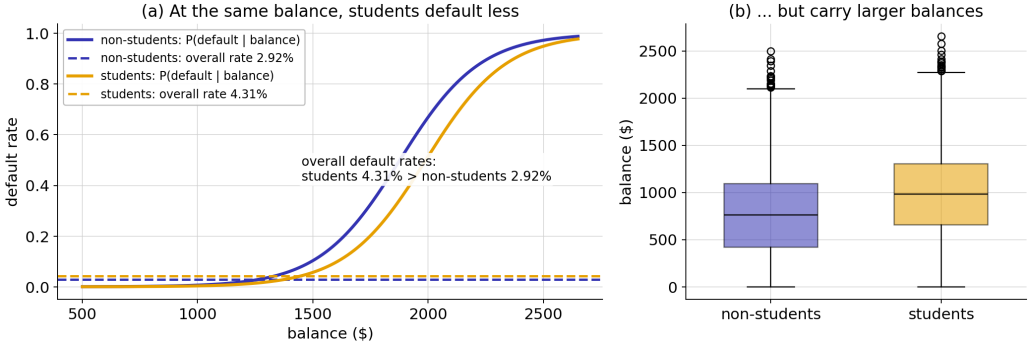
$\hat{\beta}_{\text{student}} = -0.647$ ($p = 0.006$)
At the *same* balance, students default *less often*.

How can both be true?

Students carry **larger balances** on average (988 \$ vs. 772 \$) – and a large balance is the main risk factor. Balance is a **confounder** of the student effect.

In words: The same phenomenon as the newspaper budget in Part 4: the simple coefficient answers a different question than the multiple one.

The student paradox visualised



- (a) For every balance, the student curve lies *below* the non-student curve; the overall rates (dashed) point the other way.
- (b) Students are concentrated at higher balances.

Which coefficient is "right"?

Both – they answer different questions

- **Simple model:** "Is a randomly chosen student riskier than a randomly chosen non-student?" – *yes* (relevant if nothing else is known about the applicant).
- **Multiple model:** "Is a student riskier than a non-student *with the same balance and income*?" – *no* (relevant for an individual credit decision, where the balance is known).

Key takeaway

Always ask *which variables are held fixed* before interpreting a coefficient – in linear and in logistic regression.

The student paradox with Python

```
print(df.groupby("student").balance.mean())    # No 771.77, Yes 987.82
print(g2.params["student[T.Yes]"])             # alone:              +0.4049
print(g3.params["student[T.Yes]"])             # balance, income too: -0.6468
same = pd.DataFrame({"balance": 1500, "income": 40000, "student": ["No", "Yes"]})
print(g3.predict(same))                        # same balance: 0.1050 vs 0.0579
grid = pd.DataFrame({"balance": np.linspace(500, 2650, 100), "income": 40000})
for st in ["No", "Yes"]:                       # curves as in panel (a)
    plt.plot(grid.balance, g3.predict(grid.assign(student=st)), label=st)
plt.legend()
```

- **Confounder check:** students carry larger balances (`groupby`), and balance drives default.
- **Same balance and income** (1500 \$, 40 000 \$): a non-student defaults with $\hat{p} = 0.105$, a student with only 0.058 – the multiple model compares like with like.
- `grid.assign(student=st)` sets the student column for all rows of the grid.

▶ **Try it yourself:** notebook, section 3.2 "Confounding: the student paradox" – open in Colab

Exercise – inference in the multiple model

Problem statement

In the model `default` $\sim$ `balance + income + student`:

	$\hat{\beta}_j$	$\text{SE}(\hat{\beta}_j)$
income (per \$)	$3.033 \cdot 10^{-6}$	$8.203 \cdot 10^{-6}$
`student[T.Yes]`	-0.6468	0.2363

1. Test $H_0 : \beta_{\text{income}} = 0$ at $\alpha = 0.05$.
2. Compute a 95 % CI for the odds ratio of 10 000 \$ more income.
3. Compute the odds ratio for students and its 95 % CI. Interpret it.

Solution – inference in the multiple model

(1) $Z = \dfrac{3.033}{8.203} = 0.37 < 1.96 \Rightarrow$ keep H_0 ($p = 0.71$): given balance and student status, income adds no detectable information.

(2) Per 10 000 \$: $\hat{\beta} = 0.0303$, SE $= 0.0820$:

$$0.0303 \pm 1.96 \cdot 0.0820 = [-0.130\,;\, 0.191] \;\Rightarrow\; \text{OR} \in [e^{-0.130};\, e^{0.191}] = [0.878\,;\, 1.211]$$

The interval contains 1 ("no change of the odds") – consistent with (1).

(3) $e^{-0.6468} = \mathbf{0.524}$; $-0.6468 \pm 1.96 \cdot 0.2363 = [-1.110\,;\, -0.184] \Rightarrow \text{OR} \in [0.330\,;\, 0.832]$.

Interpretation

At the same balance and income, a student's odds of default are about half (48 % lower) those of a non-student; the CI excludes 1.

Exercise – logistic regression from A to Z

Problem statement

Use the multiple model with (rounded) coefficients

$$\hat{\eta} = -10.869 + 0.00574 \cdot \text{balance} + 0.003 \cdot \text{income} - 0.6468 \cdot \text{student}$$

(income in 1000 \$, student $= 1$ for students). Two customers:

- **A**: non-student, balance 1000 \$, income 40 000 \$;
- **B**: student, balance 2000 \$, income 30 000 \$.

1. Compute $\hat{\eta}$ and $\hat{p}$ for A and B.
2. By which factor do the odds change for 100 \$ more balance?
3. At which balance does a non-student with 40 000 \$ income reach $\hat{p} = 0.5$? And a student with the same income?

Solution (1/2) – logistic regression from A to Z

(1) Log-odds and probabilities (watch the units: income in 1000 $)

$$\hat{\eta}_A = -10.869 + 0.00574 \cdot 1000 + 0.003 \cdot 40 = -10.869 + 5.740 + 0.120 = -5.009$$
$$\hat{\eta}_B = -10.869 + 0.00574 \cdot 2000 + 0.003 \cdot 30 - 0.6468 = 0.054$$

$$\hat{p}_A = \frac{e^{-5.009}}{1 + e^{-5.009}} = \mathbf{0.0066}, \qquad \hat{p}_B = \frac{e^{0.054}}{1 + e^{0.054}} = \mathbf{0.514}$$

A is a very safe customer, B is right at the knife edge. (Python with unrounded coefficients, `g3.predict(...)`: 0.0066 and 0.512.)

(2) $e^{0.00574 \cdot 100} = e^{0.574} = \mathbf{1.78}$: 100 $ more balance raises the odds by 78 % – whatever the starting point.

▶ **Try it yourself:** notebook, section 3.3 "Exercises" – open in Colab

Solution (2/2) – logistic regression from A to Z

(3) Decision boundary: $\hat{p} = 0.5 \Leftrightarrow \hat{\eta} = 0$

Non-student, income 40:

$$0 = -10.869 + 0.00574 \cdot \text{balance} + 0.120 \;\Rightarrow\; \text{balance} = \frac{10.869 - 0.120}{0.00574} \approx \mathbf{1873}\,\$$$

Student, income 40:

$$\text{balance} = \frac{10.869 - 0.120 + 0.6468}{0.00574} \approx \mathbf{1985}\,\$$$

In words: A student needs about $0.6468/0.00574 \approx 113\,\$$ more balance to reach the same 50 % risk – the negative student coefficient, expressed in dollars.

Common mistake

Inserting the income in dollars: $0.003 \cdot 40\,000 = 120$ would push every probability to 1.

Overview

1 Classification Problems
2 Logistic Regression
3 Multiple Logistic Regression
4 From Probabilities to Decisions
5 Evaluating a Classifier
6 Summary

The classification rule

Rule with threshold t

$$\hat{y} = \begin{cases} 1 \text{ (“default”)} & \text{if } \hat{p}(x) > t, \\ 0 \text{ (“no default”)} & \text{otherwise.} \end{cases}$$

The usual default is $t = 0.5$: assign the more probable class.

Threshold on the log-odds scale

Because the logistic function is increasing,

$$\hat{p}(x) > t \quad \Longleftrightarrow \quad \hat{\eta}(x) > \log \frac{t}{1-t}$$

$t = 0.5$ corresponds to $\hat{\eta} > 0$; $t = 0.2$ to $\hat{\eta} > \log(0.25) = -1.386$.

Decision boundary – default and balance

Model: $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$.

Threshold	log-odds cut-off	balance cut-off
$t = 0.5$	$\hat{\eta} > 0$	$10.65/0.0055 = 1936\,\$$
$t = 0.2$	$\hat{\eta} > \log(0.2/0.8) = -1.386$	$(10.65 - 1.386)/0.0055 = 1684\,\$$

In words: Lowering the threshold moves the boundary to the left: more defaulters are caught – but more good customers are rejected as well.

Key takeaway

The threshold is not a statistical but a **business decision**: it depends on the cost of a missed defaulter compared with the cost of a rejected good customer.

The classification rule with Python

Decision boundary on the balance scale (model g1):

```
b0, b1 = g1.params                          # -10.6513, 0.005499
print(-b0 / b1)                             # t = 0.5: boundary 1937 $
print((np.log(0.2 / 0.8) - b0) / b1)        # t = 0.2: boundary 1685 $
```

From probabilities to classes:

```
phat1 = g1.predict(df)                      # P(default) for every customer
yhat = (phat1 > 0.5).astype(int)            # rule: 1 = "default" if p-hat > t
print(yhat.sum(), (phat1 > 0.2).sum())      # flagged: 142 (t = 0.5), 462 (t = 0.2)
```

- Unrounded coefficients: boundaries 1937 and 1685 $ (previous slide, rounded: 1936 and 1684 $).
- Lowering t from 0.5 to 0.2 flags 462 instead of 142 customers. How many of them really default? → Section 5.

▶ **Try it yourself:** notebook, section 4 "From Probabilities to Decisions" – open in Colab

Exercise – from probability to decision

Problem statement

Use $\hat{\eta} = -10.65 + 0.0055 \cdot \text{balance}$. Three applicants have balances of 1800, 2000 and 2200 \$.

1. Compute the predicted probability of default for each.
2. Classify them with the threshold $t = 0.5$.
3. The bank lowers the threshold to $t = 0.2$. Which classifications change?

Solution – from probability to decision

(1) Probabilities

$$
\begin{aligned}
1800\text{:}\quad & \hat{\eta} = -10.65 + 9.90 = -0.75, && \hat{p} = \frac{e^{-0.75}}{1+e^{-0.75}} = 0.321 \\
2000\text{:}\quad & \hat{\eta} = -10.65 + 11.00 = 0.35, && \hat{p} = \frac{e^{0.35}}{1+e^{0.35}} = 0.587 \\
2200\text{:}\quad & \hat{\eta} = -10.65 + 12.10 = 1.45, && \hat{p} = \frac{e^{1.45}}{1+e^{1.45}} = 0.810
\end{aligned}
$$

(2) $t = 0.5$ (boundary 1936 \$): 1800 $\rightarrow$ **No**, 2000 $\rightarrow$ **Yes**, 2200 $\rightarrow$ **Yes**.

(3) $t = 0.2$ (boundary 1684 \$): the applicant with 1800 \$ ($\hat{p} = 0.321 > 0.2$) flips to **Yes**; the other two stay **Yes**.

In words: Equally spaced balances (+200 \$) give unequal probability steps (+0.27, +0.22) – the S-curve is non-linear.

Overview

1 Classification Problems

2 Logistic Regression

3 Multiple Logistic Regression

4 From Probabilities to Decisions

5 Evaluating a Classifier
- Confusion matrix
- Base rates and precision
- ROC curve and AUC
- Training and test data

6 Summary

Why accuracy alone is not enough

The trivial classifier

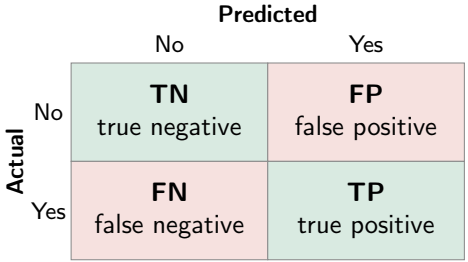
Only 3.33 % of the customers default. The "classifier" that **always predicts "no default"** is right for 96.67 % of the customers – and detects not a single defaulter.

Imbalanced classes

With a rare class, a high accuracy is easy to achieve and says little. We must look at the two kinds of errors **separately** – they also have very different costs.

In words: As in Part 3 there are two ways to be wrong: a false alarm (Type I) and a miss (Type II).

The confusion matrix



Measures

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{n}$$

$$\text{Sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- **Sensitivity** (recall): share of actual defaulters that are caught. **Specificity**: share of non-defaulters correctly left alone.
- **Precision**: share of flagged customers that really default. Error rate $= 1 -$ accuracy.

Confusion matrix – Default data, $t = 0.5$

Model: `default` $\sim$ `balance + income + student`, $t = 0.5$, all $n = 10\,000$ customers:

	Pred. No	Pred. Yes	Total
Actual No	9627	40	9667
Actual Yes	228	105	333
Total	9855	145	10 000

Metrics

Accuracy $= \frac{9627+105}{10\,000} = 0.973$ Sensitivity $= \frac{105}{333} = 0.315$

Specificity $= \frac{9627}{9667} = 0.996$ Precision $= \frac{105}{145} = 0.724$

In words: An accuracy of 97.3 % – but 228 of 333 defaulters (68 %) are missed.

The confusion matrix is a two-way table

Rows: actual class y; columns: predicted class $\hat{y}$. Every measure is a relative frequency in this table – the only question is **which total we divide by**:

Measure	**as a probability**	**divide by the ...**	**Default, $t = 0.5$**
Sensitivity	$P(\hat{y} = 1 \mid y = 1)$	row total "Actual Yes"	$105/333 = 0.315$
Specificity	$P(\hat{y} = 0 \mid y = 0)$	row total "Actual No"	$9627/9667 = 0.996$
Precision	$P(y = 1 \mid \hat{y} = 1)$	column total "Pred. Yes"	$105/145 = 0.724$
Accuracy	$P(\hat{y} = y)$	grand total n	$9732/10\,000 = 0.973$

Key takeaway

Sensitivity and precision both count the TP but condition on different events: $P(\hat{y} = 1 \mid y = 1) \neq P(y = 1 \mid \hat{y} = 1)$ – like $P(\text{default} \mid \text{student}) \neq P(\text{student} \mid \text{default})$.

Rates with Python: normalise the rows or the columns

```
phat = g3.predict(df)                                          # P(default = Yes), model g3
pred = phat > 0.5                                              # True = "default" predicted
print(pd.crosstab(df.default, pred, normalize="index").round(4))     # given y
print(pd.crosstab(df.default, pred, normalize="columns").round(4))   # given y-hat
print((pred == df.y).mean())                                   # accuracy 0.9732
```

```
col_0      False   True
default
No        0.9959  0.0041
Yes       0.6847  0.3153
col_0      False   True
default
No        0.9769  0.2759
Yes       0.0231  0.7241
0.9732
```

- **Rows** sum to 1: row "Yes" gives the sensitivity 0.3153 (and the miss rate 0.6847), row "No" the specificity 0.9959.
- **Columns** sum to 1: column "True" gives the precision 0.7241; column "False": 97.7 % of the customers predicted "no default" really do not default.
- **Accuracy**: the share of customers with $\hat{y} = y$.

▶ **Try it yourself:** notebook, section 5.1 "Confusion matrix" – open in Colab

Exercise – reading a confusion matrix

Problem statement

The bank lowers the threshold of the same model to $t = 0.2$:

	Pred. No	Pred. Yes	Total
Actual No	9390	277	9667
Actual Yes	130	203	333

1. Compute accuracy, error rate, sensitivity, specificity and precision.
2. Compare with the confusion matrix at $t = 0.5$. Which threshold would you recommend to the bank?

Solution – reading a confusion matrix

(1) Metrics at $t = 0.2$ (with $t = 0.5$ for comparison):

	$t = 0.5$	$t = 0.2$	
Accuracy	0.973	0.959	$= (9390 + 203)/10\,000$
Error rate	0.027	0.041	$= 1 - 0.959$
Sensitivity	0.315	0.610	$= 203/333$
Specificity	0.996	0.971	$= 9390/9667$
Precision	0.724	0.423	$= 203/(203 + 277)$

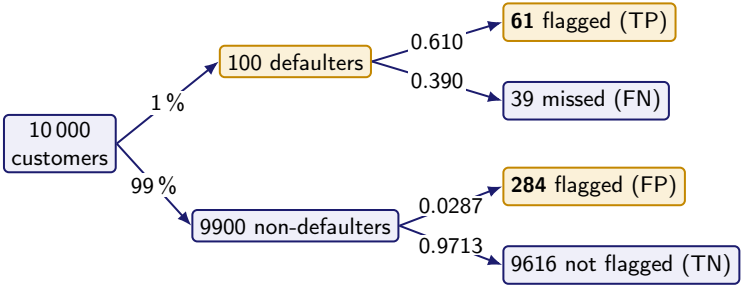
(2) With $t = 0.2$ the bank catches almost twice as many defaulters (203 instead of 105) at the price of 237 additional false alarms. If a missed default costs much more than a rejected good customer, $t = 0.2$ is the better choice – although its accuracy is lower.

In words: Do not confuse sensitivity and precision: both have TP on top, but divide by the *actual* (333) or the *predicted* (480) positives.

The base-rate effect

Guiding question

Our rule with $t = 0.2$ catches 61 % of the defaulters and flags only 2.87 % of the good customers. Are its alarms just as reliable in a bank where only 1 % of the customers default?



Precision

$$\frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{61}{61 + 284} = \mathbf{0.177}$$

Only about one alarm in six – compared with 0.423 at the actual base rate of 3.33 %.

In words: The rule has not changed (same sensitivity and specificity) – the mix has: with rare defaulters, the few false alarms from the huge majority outnumber the correct alarms.

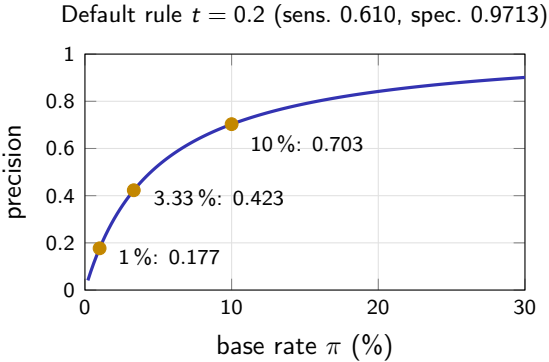
Precision depends on the base rate

Precision for base rate $\pi = P(Y = 1)$

$$\text{precision} = \frac{\text{sens} \cdot \pi}{\text{sens} \cdot \pi + (1 - \text{spec})(1 - \pi)}$$

In words: correct alarms divided by all alarms; false alarms come from the $1 - \pi$ non-defaulters.

Check ($\pi = 0.0333$): $0.610 \cdot 0.0333 = 0.0203$, $0.0287 \cdot 0.9667 = 0.0277 \Rightarrow$ $0.0203/0.0480 = 0.423$ ✓



Key takeaway

Sensitivity and specificity describe the *rule*; precision also depends on the **base rate** (Appendix A.1).

Base rates with Python

```
def precision(pi, sens, spec):
    """Share of correct alarms among all alarms at base rate pi."""
    return sens * pi / (sens * pi + (1 - spec) * (1 - pi))

sens, spec = 203 / 333, 9390 / 9667          # rule t = 0.2: 0.6096, 0.9713
print(round(10000 * 0.01 * sens), round(10000 * 0.99 * (1 - spec)))  # 61 TP, 284 FP
for pi in [0.01, 0.0333, 0.10]:
    print(pi, round(precision(pi, sens, spec), 3))       # 0.177  0.423  0.703
```

```
61 284
0.01 0.177
0.0333 0.423
0.1 0.703
```

- The function is the formula of the previous slide; a `for` loop evaluates it for several base rates.
- Natural frequencies: 10 000 customers × base rate × conditional rate – the counts of the tree.

▶ **Try it yourself:** notebook, section 5.2 "Base rates and precision" – open in Colab

Exercise – rapid test and base rate

Problem statement

A rapid test is designed to detect a disease that affects 1 % of the population (**prevalence** = base rate). Sensitivity: an ill person tests positive with probability 0.9. Specificity: a healthy person tests negative with probability 0.9.

1. Set up the expected confusion matrix for 1000 people tested.
2. What share of all tests is positive?
3. A randomly chosen person tests positive. How likely is the person ill (precision)? Interpret the result.
4. What share of the negative results is correct?
5. In a doctor's practice, 20 % of the people tested are ill. Recompute (3) for this group.

The same calculation applies to quality control (defective part $\leftrightarrow$ alarm) and to spam filters.

Solution (1/2) – rapid test and base rate

Rule: expected count $= n \times$ base rate $\times$ conditional rate ("natural frequencies").

(1) Of 1000 people, $1000 \cdot 0.01 = 10$ are ill and 990 healthy. Of the ill, $0.9 \cdot 10 = 9$ test positive; of the healthy, $0.9 \cdot 990 = 891$ test negative and 99 positive.

	test −	test +	Total
healthy	891 (TN)	99 (FP)	990
ill	1 (FN)	9 (TP)	10
Total	892	108	1000

(2) $P(+) = \dfrac{108}{1000} = \mathbf{0.108}$: 10.8 % of all tests are positive – and 99 of these 108 come from healthy people.

Solution (2/2) – rapid test and base rate

(3) Condition on the column "test +": $P(\text{ill} \mid +) = \dfrac{\text{TP}}{\text{TP} + \text{FP}} = \dfrac{9}{108} = \mathbf{0.083}$.

Only one in twelve positive results is a real case: the 99 false alarms (10 % of the 990 healthy) outnumber the 9 correct ones. Still, the test is informative: the probability rises from 1 % to 8.3 %.

(4) Condition on the column "test −": $\frac{891}{892} = \mathbf{0.9989}$ – a negative result is very reliable.

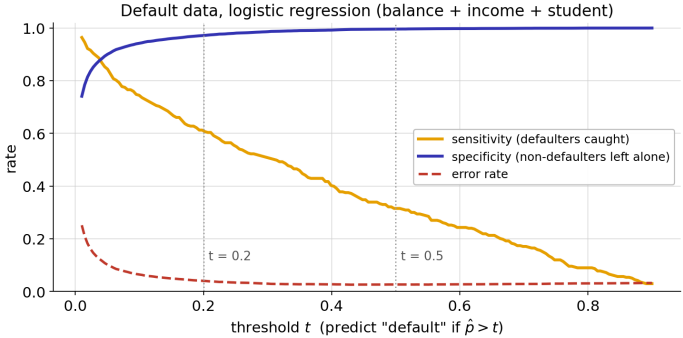
(5) 1000 people: 200 ill $\to 0.9 \cdot 200 = 180$ positive, 800 healthy $\to 0.1 \cdot 800 = 80$ positive. Precision $= 180/(180 + 80) = 180/260 = \mathbf{0.692}$.

Key takeaway

Same test, different base rate – a completely different meaning of a positive result. Python check: `precision(0.01, 0.9, 0.9)` and `precision(0.20, 0.9, 0.9)` give 0.083 and 0.692.

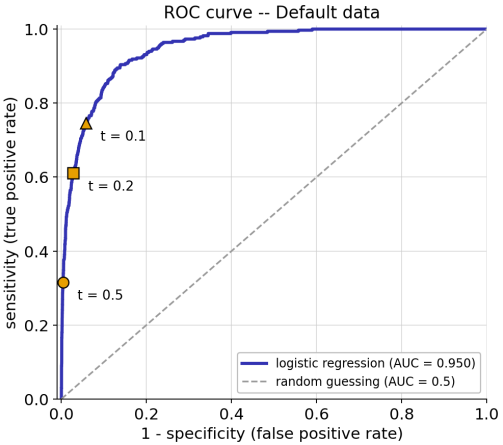
▶ **Try it yourself:** notebook, section 5.2 "Base rates and precision" – open in Colab

The threshold trade-off



- Lower $t \Rightarrow$ sensitivity $\uparrow$, specificity $\downarrow$ – and vice versa.
- The error rate is minimal around $t = 0.5$, but that is not necessarily the *cheapest* threshold.

The ROC curve



Construction

For *every* threshold t, plot the pair

$$\big(1 - \text{specificity},\ \text{sensitivity}\big).$$

- Each point is one threshold; the curve shows all of them.
- The diagonal is random guessing.
- The closer the curve gets to the **top-left corner**, the better the classifier.

AUC – area under the ROC curve

Definition and interpretation

The AUC is the area under the ROC curve. Equivalently,

$$\text{AUC} = P\big(\hat{p}(\text{random defaulter}) > \hat{p}(\text{random non-defaulter})\big).$$

- AUC $= 0.5$: no better than random; AUC $= 1$: perfect ranking.
- **Default data:** AUC $= 0.950$ – a randomly chosen defaulter receives a higher predicted probability than a randomly chosen non-defaulter in 95 % of all pairs.
- The AUC summarises **all thresholds** at once – changing t moves along the curve but never changes the AUC.

Key takeaway

The ROC curve and AUC evaluate the *model*; the confusion matrix evaluates one *decision rule* (one threshold).

ROC curve and AUC with Python

Sensitivity and specificity for several thresholds (`phat` from the model g3):

```
for t in [0.1, 0.2, 0.5]:           # sensitivity, specificity
    print(t, (phat[df.y == 1] > t).mean(), (phat[df.y == 0] <= t).mean())
# 0.1: 0.7447 0.9421   0.2: 0.6096 0.9713   0.5: 0.3153 0.9959
```

ROC curve and AUC:

```
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thr = roc_curve(df.y, phat)   # one point per threshold
plt.plot(fpr, tpr)                      # ROC curve
plt.plot([0, 1], [0, 1], "--")          # diagonal: random guessing
plt.xlabel("1 - specificity"); plt.ylabel("sensitivity")
print(roc_auc_score(df.y, phat))        # AUC 0.950
```

In words: `phat[df.y == 1]` keeps the defaulters; the mean of `> t` is the share of them that are flagged.

▶ **Try it yourself:** notebook, section 5.3 "ROC curve and AUC" – open in Colab

Exercise – threshold, ROC and AUC

Problem statement

For the Default model: $t = 0.5$ gives sensitivity 0.315 and specificity 0.996; $t = 0.2$ gives sensitivity 0.610 and specificity 0.971.

1. Locate both operating points on the ROC axes.
2. How does the point move when the threshold drops from 0.5 to 0.2?
3. A colleague hopes to improve the AUC by choosing a better threshold. Comment.

Solution – threshold, ROC and AUC

(1) $t = 0.5$: $(1 - 0.996,\ 0.315) = (0.004,\ 0.315)$;
$t = 0.2$: $(1 - 0.971,\ 0.610) = (0.029,\ 0.610)$.

(2) The point moves **up and to the right along the same curve**: a small loss in specificity ($+0.025$ on the x-axis) buys a large gain in sensitivity ($+0.295$).

(3) Impossible: the threshold only selects a point *on* the curve. The AUC belongs to the model's ranking of the customers – only a better *model* (better variables) can raise it.

Practice

Choose the model by its AUC, then choose the threshold by the costs of the two error types.

Honest evaluation: training and test data

Problem

So far we evaluated the model on the *same* data used to fit it. This is **optimistic**: the model has adapted to exactly these customers. What counts is the performance on **new** customers.

Solution: split the data

- **Training data** (e.g. 70 %): estimate the model.
- **Test data** (the remaining 30 %): compute the confusion matrix and the AUC – data the model has never seen.

Default data: 7000 training, 3000 test customers (`random_state=1`)

At $t = 0.5$ the model catches 34 of 112 test defaulters, at $t = 0.2$ already 67. Test AUC $= 0.956$ (in-sample on all data: 0.950) – with only four coefficients there is little to overfit, and chance can even push a test value above the in-sample one.

Exercise – training vs. test data

Problem statement

The model `default ~ balance + income + student` is fitted on 7000 training customers and applied to the 3000 test customers (112 of them defaulted):

	TN	FP	FN	TP
Test data, $t = 0.5$	2878	10	78	34
Test data, $t = 0.2$	2824	64	45	67

1. Compute accuracy, sensitivity, specificity and precision for both thresholds.
2. On all 10 000 customers (training = evaluation data) the sensitivity at $t = 0.5$ was 0.315. Compare and explain.

Solution – training vs. test data

(1) Test-data metrics:

	$t = 0.5$	$t = 0.2$
Accuracy	$\frac{2878+34}{3000} = 0.971$	$\frac{2824+67}{3000} = 0.964$
Sensitivity	$\frac{34}{112} = 0.304$	$\frac{67}{112} = 0.598$
Specificity	$\frac{2878}{2888} = 0.997$	$\frac{2824}{2888} = 0.978$
Precision	$\frac{34}{44} = 0.773$	$\frac{67}{131} = 0.511$

(2) On new customers the model catches 30.4 % of the defaulters at $t = 0.5$, slightly less than the 31.5 % on the data it was fitted on. In-sample figures tend to be **optimistic**; here the gap is small (a simple model, 7000 training customers), and with only 112 test defaulters part of any difference is chance.

In words: Report the test-data metrics – they describe what the bank can expect for future applicants.

Classification with Python

Predicted probabilities and confusion matrix

```
phat = g3.predict(df)              # P(default = Yes); log-odds: which="linear"
print(pd.crosstab(df.default, phat > 0.5))     # t = 0.5: 9627 40 / 228 105
```

Training and test data, test AUC

```
from sklearn.metrics import roc_auc_score
train = df.sample(n=7000, random_state=1)      # 70 % training data
test = df.drop(train.index)                    # remaining 30 % test data
g_tr = smf.logit("y ~ balance + income + student", data=train).fit(disp=0)
p_te = g_tr.predict(test)                      # test-set probabilities
print(pd.crosstab(test.default, p_te > 0.5))   # 2878 10 / 78 34
print(roc_auc_score(test.y, p_te))             # test AUC 0.956
```

▶ **Try it yourself:** notebook, section 5.4 "Training and test data" – open in Colab

Overview

1 Classification Problems
2 Logistic Regression
3 Multiple Logistic Regression
4 From Probabilities to Decisions
5 Evaluating a Classifier
6 Summary

Summary 1 – two-way tables and conditional probability

Two-way table of two categorical variables

joint relative frequency: cell/n; **marginal**: row or column total/n; **conditional**: cell/row (or column) total.

Rules

$P(A \mid B) = P(A \cap B)/P(B)$

$P(A \cap B) = P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$

independent:

$P(A \cap B) = P(A)\,P(B) \Leftrightarrow P(A \mid B) = P(A)$

Link to classification

Classification estimates $p(x) = P(Y = 1 \mid X = x)$ – for a metric x with a model. Sensitivity $P(\hat{y} = 1 \mid y = 1)$ and precision $P(y = 1 \mid \hat{y} = 1)$ condition in opposite directions.

In words: $P(A \mid B) \neq P(B \mid A)$: 4.3 % of the students default, but 38 % of the defaulters are students.

Summary 2 – logistic regression

Model

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p, \qquad p(x) = \frac{e^\eta}{1+e^\eta}$$

estimated by **maximum likelihood**.

Interpretation of β_j (holding the other variables fixed)

- sign: direction of the effect on the probability,
- β_j: change of the log-odds per unit of x_j,
- e^{β_j}: **odds ratio** – factor by which the odds change per unit of x_j.

Inference

$Z = \hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j) \approx N(0,1)$, CI $\hat{\beta}_j \pm z_{1-\alpha/2}\mathrm{SE}(\hat{\beta}_j)$, CI for the odds ratio: exponentiate the bounds.

Summary 3 – decisions and evaluation

Classification rule

$\hat{y} = 1$ if $\hat{p}(x) > t \;\Leftrightarrow\; \hat{\eta}(x) > \log \frac{t}{1-t}$; the threshold t is a business decision.

Measure	Formula	Probability	Question
Accuracy	$(\mathrm{TP} + \mathrm{TN})/n$	$P(\hat{y} = y)$	share of correct decisions
Sensitivity	$\mathrm{TP}/(\mathrm{TP} + \mathrm{FN})$	$P(\hat{y} = 1 \mid y = 1)$	share of positives caught
Specificity	$\mathrm{TN}/(\mathrm{TN} + \mathrm{FP})$	$P(\hat{y} = 0 \mid y = 0)$	share of negatives left alone
Precision	$\mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$	$P(y = 1 \mid \hat{y} = 1)$	share of alarms that are real
AUC	area under ROC	–	ranking over all thresholds

In words: precision also depends on the base rate π: $\mathrm{sens} \cdot \pi \,/\, [\mathrm{sens} \cdot \pi + (1 - \mathrm{spec})(1 - \pi)]$. Evaluate on **test data** the model has not seen.

Summary 4 – important Python commands

Two-way tables

```
pd.crosstab(df.a, df.b, margins=True)
pd.crosstab(df.a, df.b,
            normalize="index")  # rows
```

Logistic regression

```
g = smf.logit("y ~ x1 + x2",
              data=df).fit()
print(g.summary())     # z-tests
np.exp(g.params)       # odds ratios
np.exp(g.conf_int())   # CIs for ORs
g.predict(new)         # probabilities
```

Decisions and evaluation

```
phat = g.predict(df)
pd.crosstab(df.y, phat > 0.5)
pd.crosstab(df.y, phat > 0.5,
            normalize="index")
roc_auc_score(df.y, phat)  # AUC
train = df.sample(n=7000,
                  random_state=1)
test = df.drop(train.index)
```

→ *Appendix: Bayes classifiers in sklearn: `LinearDiscriminantAnalysis`, `GaussianNB`*

Six classic pitfalls – final checklist

Forbidden sentence ×	**Correct version** ✓
"The odds ratio is 1.78, so the probability rises by 78 %."	The *odds* rise by 78 %; the change in probability depends on the starting point.
"Accuracy 97 % – an excellent classifier."	With 3 % defaulters, always saying "no" gives 97 %; check sensitivity and specificity.
"$t = 0.5$ is the correct threshold."	The threshold depends on the costs of the two errors.
"A better threshold improves the AUC."	The threshold moves along the ROC curve; only a better model raises the AUC.
"Students are riskier, so reject students."	At the same balance, students are *less* risky – ask what is held fixed.
"The test catches 95 % of the defects, so 95 % of the flagged parts are defective."	That is $P(+ \mid D)$; $P(D \mid +)$ also depends on the base rate – with 2 % defects only 33 %.

Looking back: the lecture in one slide

Part	Key idea
1 – Descriptive Statistics	Summarise data by location and dispersion ($\bar{x}$, $\tilde{x}$, s, v).
2 – Graphical Representations	Show the distribution: empirical CDF, histogram, boxplot.
3 – Statistical Inference	Infer from sample to population: estimate $\pm$ critical value $\times$ SE; tests and p-values.
4 – Regression	Explain a metric Y by several X – with t-tests, CIs and diagnostics.
5 – Classification	Explain a categorical Y: logistic regression, decision rules, evaluation.

Key takeaway

One thread through all five parts: every number computed from data is an **estimate** – describe it, picture it, and report its uncertainty.

Overview

7 Appendix: Bayes' Theorem and Generative Classifiers
- A.1 Bayes' theorem
- A.2 The Bayes classifier
- A.3 Naive Bayes
- A.4 Linear discriminant analysis
- A.5 Comparison

Appendix – what is in here, and why

Optional material

The main part covers the standard classification workflow with logistic regression. The appendix first derives **Bayes' theorem** – the rule behind the base-rate effect of Section 5 – and then shows a second family of classifiers built on it, useful for small samples, many features or more than two classes.

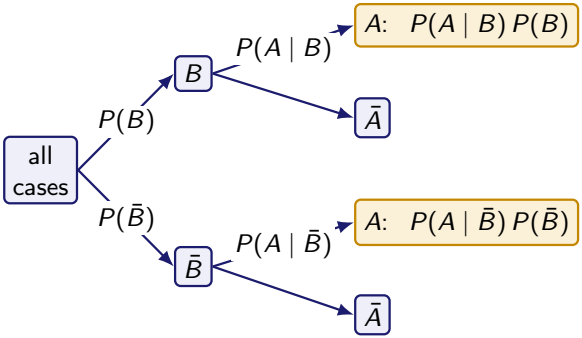
Topic	**Content**
A.1 Bayes' theorem	Total probability, prior $\times$ likelihood $\rightarrow$ posterior, taxi example; exercises: suppliers, quality test
A.2 Bayes classifier	Bayes' theorem with classes and features; largest posterior wins
A.3 Naive Bayes	Independent features: the spam filter (exercise)
A.4 LDA	Normal classes with common variance (exercise)
A.5 Comparison	Logistic regression, LDA and naive Bayes; Python commands

Law of total probability

Two cases B and $\bar{B}$ ("not B")

$$P(A) = P(A \mid B)\,P(B) + P(A \mid \bar{B})\,P(\bar{B})$$

In general, if $B_1, \ldots, B_k$ are disjoint and together cover all cases:
$P(A) = \sum_j P(A \mid B_j)\,P(B_j)$.



In words: multiply along each path (multiplication rule) and add up **all paths that end in** A.

Example (Section 5): how often does the rapid test raise an alarm?

$$P(+) = P(+ \mid \text{ill})\,P(\text{ill}) + P(+ \mid \text{healthy})\,P(\text{healthy}) = 0.9 \cdot 0.01 + 0.1 \cdot 0.99 = 0.108$$

Bayes' theorem – reversing a conditional probability

Known: $P(+ \mid \text{ill})$. **Wanted:** $P(\text{ill} \mid +)$ – the conditional probability in the other direction.

Step 1: multiplication rule in both directions, $P(A \mid B)\,P(B) = P(A \cap B) = P(B \mid A)\,P(A)$; solve for $P(B \mid A)$.

Step 2: compute the denominator $P(A)$ with the law of total probability.

Bayes' theorem

$$\underbrace{P(B_i \mid A)}_{\text{posterior}} = \frac{\overbrace{P(A \mid B_i)}^{\text{likelihood}}\,\overbrace{P(B_i)}^{\text{prior}}}{\underbrace{\sum_j P(A \mid B_j)\,P(B_j)}_{\text{total probability } P(A)}} \qquad \text{two cases:} \quad P(B \mid A) = \frac{P(A \mid B)\,P(B)}{P(A \mid B)\,P(B) + P(A \mid \bar{B})\,P(\bar{B})}$$

In words: observe $A \Rightarrow$ update the probabilities of the possible explanations B_i. Rapid test: $P(\text{ill}) = 0.01 \rightarrow P(\text{ill} \mid +) = 0.009/0.108 = 0.083$.

Bayes' theorem – the three building blocks

Prior $P(B_i)$

What we know about B_i *before* observing A – often a known share, e.g. the base rate of a disease or of defaulters.

Likelihood $P(A \mid B_i)$

How probable the observation A is *if* B_i is true – e.g. the sensitivity of a test.

Posterior $P(B_i \mid A)$

What we know about B_i *after* observing A – the quantity we actually want.

Bayes as a learning rule

$$P(B_i) \xrightarrow{\text{observe } A} P(B_i \mid A)$$

Prior knowledge is updated by data.

Dictionary for classification

cases B_i	classes $Y = k$
observation A	features $X = x$
prior $P(B_i)$	class share π_k
likelihood	$f_k(x)$ (A.2)

Worked example: the night-time taxi (1/2)

Scenario

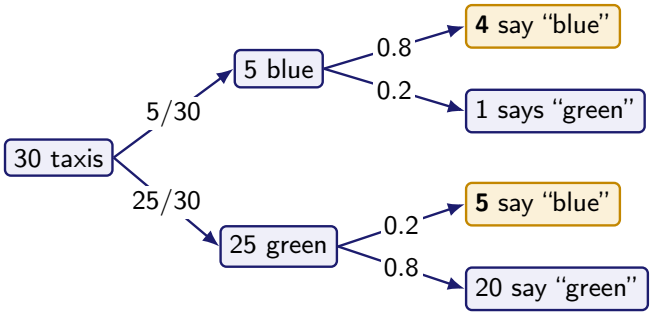
At night a taxi causes an accident. A witness says: "It was a **blue** taxi." The town has 5 blue and 25 green taxis. Tests show that the witness recognises *both* colours correctly in 80 % of the cases. How likely was the taxi really blue?

Events: B = the taxi was really blue, W = the witness says "blue".

Prior: $P(B) = 5/30$, $P(\bar{B}) = 25/30$.

Likelihoods: $P(W \mid B) = 0.8$, $P(W \mid \bar{B}) = 0.2$ (green mistaken for blue).

Tree: natural frequencies – imagine the scene once with each of the 30 taxis.



Worked example: the night-time taxi (2/2)

Total probability: when does the witness say "blue"?

$$P(W) = \underbrace{0.8 \cdot \tfrac{5}{30}}_{\text{blue, correct}} + \underbrace{0.2 \cdot \tfrac{25}{30}}_{\text{green, mistaken}} = \tfrac{4}{30} + \tfrac{5}{30} = \tfrac{9}{30} = 0.3$$

Bayes' theorem

$$P(B \mid W) = \frac{0.8 \cdot \frac{5}{30}}{\frac{9}{30}} = \frac{4}{9} = \mathbf{0.444}$$

	says "blue"	says "green"	Total
really blue	**4**	1	5
really green	5	20	25
Total	**9**	21	30

counts out of 30 scenes ($\div 30$ = probabilities)

In words: only 4 of the 9 "blue" statements are correct – the other 5 come from the many green taxis. The witness raises the probability from $\frac{5}{30} = 17\,\%$ (prior) to 44 % (posterior) – still less than 50 %.

Bayes' theorem with Python

Night-time taxi – one array entry per case B_i (blue, green):

```
prior = np.array([5, 25]) / 30          # P(blue), P(green):          0.1667 0.8333
lik = np.array([0.8, 0.2])              # P(says "blue" | colour)
joint = prior * lik                     # multiplication rule:        0.1333 0.1667
print(joint.sum())                      # total probability P(W):     0.3
print(joint / joint.sum())              # posterior P(colour | W):    0.4444 0.5556
```

The same steps as a function – rapid test of Section 5:

```
def bayes(prior, lik):
    joint = prior * lik                 # prior x likelihood
    return joint / joint.sum()          # divided by the total probability
print(bayes(np.array([0.01, 0.99]), np.array([0.9, 0.1])))   # P(ill | +) 0.0833
```

In words: works for any number of cases: put all priors and likelihoods into arrays; the posteriors always add up to 1.

▶ **Try it yourself:** notebook, section A.1 "Bayes' theorem" – open in Colab

Exercise – three suppliers

Problem statement

An electronics manufacturer buys a component from three suppliers: L_1 delivers 50 %, L_2 30 % and L_3 20 % of the parts. Incoming inspection shows that 2 % of the parts from L_1 are defective (D), 4 % from L_2 and 10 % from L_3. A part is taken at random from stock.

1. Write the given numbers as probabilities $P(L_i)$ and $P(D \mid L_i)$.
2. Compute $P(L_i \cap D)$ for $i = 1, 2, 3$.
3. Compute $P(D)$. How many defective parts are expected among 1000?
4. Are "defective" and "from L_1" independent?
5. The part is defective. Compute $P(L_i \mid D)$ for all three suppliers. Which supplier is the most likely source?

Solution (1/2) – three suppliers

(1) L_1, L_2, L_3 split all parts (each part comes from exactly one supplier), so the priors add up to 1. Given: the first two rows of the table.

(2) Multiplication rule $P(L_i \cap D) = P(D \mid L_i)\, P(L_i)$: the third row.

	L_1	L_2	L_3
prior $P(L_i)$	0.5	0.3	0.2
likelihood $P(D \mid L_i)$	0.02	0.04	0.10
$P(L_i \cap D)$	$0.02 \cdot 0.5 = \mathbf{0.010}$	$0.04 \cdot 0.3 = \mathbf{0.012}$	$0.10 \cdot 0.2 = \mathbf{0.020}$

In words: 1 % of all parts in stock are defective parts from L_1, 1.2 % from L_2 and 2 % from L_3 – the *smallest* supplier contributes the *most* defective parts.

Solution (2/2) – three suppliers

(3) Total probability: $P(D) = 0.010 + 0.012 + 0.020 = \mathbf{0.042}$; among 1000 parts we expect **42** defective ones (10 from L_1, 12 from L_2, 20 from L_3).

(4) $P(L_1)\,P(D) = 0.5 \cdot 0.042 = 0.021 \neq 0.010 = P(L_1 \cap D) \Rightarrow$ **not independent**; equivalently $P(D \mid L_1) = 0.02 \neq P(D) = 0.042$.

(5) Bayes' theorem $P(L_i \mid D) = P(L_i \cap D)/P(D)$:

$$P(L_1 \mid D) = \frac{0.010}{0.042} = \mathbf{0.238}, \quad P(L_2 \mid D) = \frac{0.012}{0.042} = \mathbf{0.286}, \quad P(L_3 \mid D) = \frac{0.020}{0.042} = \mathbf{0.476}$$

The posteriors add up to 1. The most likely source is $\mathbf{L_3}$ (largest posterior – the rule of the Bayes classifier in A.2): although L_3 delivers only 20 % of the parts, its high defect rate (likelihood) outweighs the small prior.

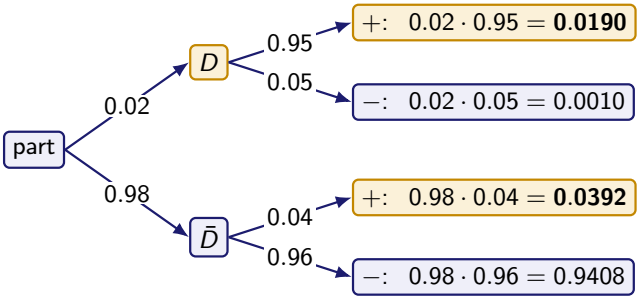
Exercise – automated quality test (exam style)

Problem statement

In a production line 2 % of all parts are defective (D). An automated test raises an alarm ($+$) for a defective part with probability 0.95 and – wrongly – for an intact part with probability 0.04.

1. Draw a probability tree and compute $P(D \cap +)$ and $P(\bar{D} \cap +)$.
2. Compute the probability $P(+)$ that the test raises an alarm.
3. Compute $P(D \mid +)$. Explain in one sentence why only about a third of the flagged parts are defective, although the test catches 95 % of them.
4. In the language of Section 5: what are the base rate, sensitivity, specificity and precision of this test?

Solution (1/2) – automated quality test (exam style)



(1) Path rule (multiplication rule):

$P(D \cap +) = 0.02 \cdot 0.95 = \mathbf{0.0190}$,

$P(\bar{D} \cap +) = 0.98 \cdot 0.04 = \mathbf{0.0392}$.

The four path probabilities add up to 1.

(2) Total probability – add both paths that end in "+":

$$P(+) = P(+ \mid D)\,P(D) + P(+ \mid \bar{D})\,P(\bar{D}) = 0.0190 + 0.0392 = \mathbf{0.0582}$$

The larger part of the alarms (0.0392 of 0.0582) comes from *intact* parts.

Solution (2/2) – automated quality test (exam style)

(3) Bayes' theorem: $P(D \mid +) = \dfrac{P(+ \mid D)\,P(D)}{P(+)} = \dfrac{0.0190}{0.0582} = \mathbf{0.3265}$.

Natural frequencies for 10 000 parts:

	no alarm	alarm	Total
intact	9408	392	9800
defective	10	190	200
Total	9418	582	10 000

Only 190 of the 582 alarms are justified: $190/582 = 0.3265$. Because defective parts are rare (2 %), the 4 % false alarms among the 98 % intact parts produce more alarms than the defective parts themselves.

(4) Base rate $\pi = 0.02$, sensitivity 0.95, specificity $1 - 0.04 = 0.96$, precision **0.3265** – the base-rate effect of Section 5.

Common mistake

Confusing $P(D \mid +)$ with $P(+ \mid D)$: "95 % of the flagged parts are defective" is wrong.

Two ways to obtain $P(Y = k \mid x)$

Discriminative

Model $P(Y = 1 \mid x)$ **directly**.

Example: logistic regression.

Generative

Model how x is distributed **within each class** and combine it with the class frequencies via **Bayes' theorem**.

Examples: naive Bayes, LDA.

When generative models are useful

Small samples, well separated classes, more than two classes – and they make the **base rate** (prior) explicit, which matters whenever one class is rare.

Bayes' theorem in the classification setting

Posterior of class k (Bayes' theorem, A.1, with $B_k = \{Y = k\}$, $A = \{X = x\}$)

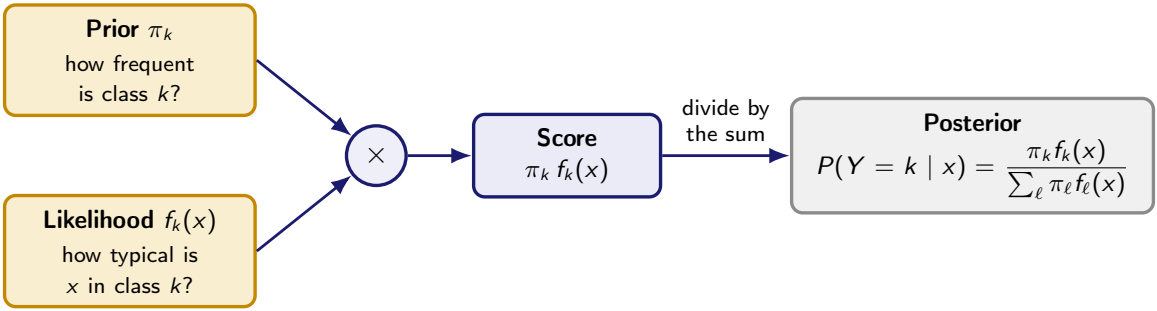
$$P(Y = k \mid X = x) = \frac{\pi_k \, f_k(x)}{\sum_\ell \pi_\ell \, f_\ell(x)}$$

Symbol	Meaning
$\pi_k = P(Y = k)$	**prior**: share of class k (e.g. 3.33 % defaulters)
$f_k(x)$	**likelihood**: how typical is x for class k?
$P(Y = k \mid x)$	**posterior**: probability of class k given x

Bayes classifier

Assign x to the class with the **largest posterior** – for two classes: $\hat{y} = 1$ if $P(Y = 1 \mid x) > 0.5$.

Bayes' theorem as a flow



Key takeaway

Every generative classifier is this flow – naive Bayes and LDA differ only in **how they model the likelihood** $f_k(x)$.

Naive Bayes: an extreme simplification

Assumption

Within each class, the explanatory variables are **independent**:

$$f_k(x) = f_{k1}(x_1) \cdot f_{k2}(x_2) \cdots f_{kp}(x_p).$$

- Instead of one p-dimensional distribution, only p one-dimensional ones have to be estimated – e.g. simple proportions for yes/no features.
- The assumption is almost always *wrong* – yet the *ranking* of the cases is often good enough to classify well.

In words: Classic application: spam filters – each word contributes its own factor to the score.

Exercise – naive Bayes spam filter

Problem statement

A spam filter has priors $P(\text{Spam}) = 0.4$, $P(\text{Ham}) = 0.6$ and two binary features: F_1 ("contains the word *free*") and F_2 ("contains a link").

	$P(F_1 = 1 \mid \cdot)$	$P(F_2 = 1 \mid \cdot)$
Spam	0.6	0.7
Ham	0.1	0.2

A new e-mail contains *both* features.

1. Compute the score $\pi_k \, f_k(x)$ of each class.
2. Compute $P(\text{Spam} \mid F_1 = 1, F_2 = 1)$ and classify the e-mail.

Solution – naive Bayes spam filter

(1) Scores (prior × product of the feature probabilities)

$$\text{Spam: } 0.4 \cdot 0.6 \cdot 0.7 = 0.168, \qquad \text{Ham: } 0.6 \cdot 0.1 \cdot 0.2 = 0.012$$

(2) Posterior

$$P(\text{Spam} \mid F_1, F_2) = \frac{0.168}{0.168 + 0.012} = \frac{0.168}{0.180} = \mathbf{0.933} > 0.5 \Rightarrow \textbf{Spam}$$

In words: The two features together are 21 times as likely under Spam as under Ham (0.42/0.02); this overwhelms the prior, which favoured Ham.

Common mistake

Ignoring the prior: $0.42/(0.42 + 0.02) = 0.955$ overstates the spam probability. With rare positives the prior can even flip the decision.

Linear discriminant analysis (LDA) with one variable

Assumption

Within each class, X is **normally distributed** with mean μ_k and a **common** variance σ^2: $X \mid Y = k \sim N(\mu_k, \sigma^2)$.

Discriminant score and decision boundary (two classes)

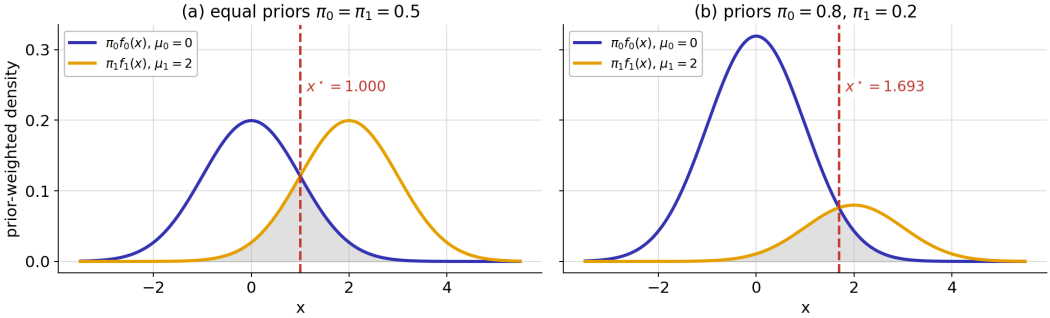
Inserting the normal densities into Bayes' theorem and taking logs gives a score **linear** in x:

$$\delta_k(x) = x\,\frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log \pi_k, \qquad x^\star = \frac{\mu_0 + \mu_1}{2} - \frac{\sigma^2}{\mu_1 - \mu_0}\,\log\frac{\pi_1}{\pi_0}.$$

Assign the class with the larger score; at $x^\star$ both scores are equal.

In words: Equal priors: the boundary is the midpoint of the means. A rare class 1 pushes the boundary towards μ_1.

LDA with one variable – in pictures



- LDA compares the **prior-weighted densities** $\pi_k f_k(x)$; the boundary lies where they cross.
- The grey overlap is the error that *no* classifier can avoid.

Exercise – LDA in one dimension

Problem statement

Two classes with $X \mid Y = k \sim N(\mu_k, \sigma^2)$, $\sigma^2 = 1$, $\mu_0 = 0$, $\mu_1 = 2$.

1. With equal priors, write down $\delta_0(x)$ and $\delta_1(x)$ and find the decision boundary.
2. Classify $x = 1.5$.
3. Now take $\pi_0 = 0.8$, $\pi_1 = 0.2$. Recompute the boundary and reclassify $x = 1.5$.

Solution – LDA in one dimension

(1)–(2) Equal priors

$\delta_0(x) = 0 + \log 0.5, \quad \delta_1(x) = 2x - 2 + \log 0.5$. The equal $\log \pi_k$ terms cancel: $0 = 2x - 2 \Rightarrow x^\star = 1$ (the midpoint of the means). $x = 1.5 > 1 \Rightarrow$ **class 1**.

(3) Unequal priors

$\delta_0 = \log 0.8 = -0.223, \quad \delta_1 = 2x - 2 + \log 0.2 = 2x - 3.609$:

$$-0.223 = 2x - 3.609 \;\Rightarrow\; x^\star = 1.693 \qquad \left(\text{check: } 1 - \tfrac{1}{2}\log\tfrac{0.2}{0.8} = 1 + 0.693 \;\checkmark\right)$$

Now $x = 1.5 < 1.693 \Rightarrow$ **class 0**.

In words: Same observation, opposite decision – purely because class 1 became rare: the rarer class needs stronger evidence.

Logistic regression, LDA and naive Bayes

	Logistic regression	**LDA**	**Naive Bayes**
Models	$P(Y = 1 \mid x)$ directly	normal f_k, common variance	independent features
Boundary	linear in x	linear in x	flexible
Strength	few assumptions; interpretable odds ratios, tests	stable for small n, several classes	very many features, small n
Weakness	unstable if classes are perfectly separated	wrong if normality fails	miscalibrated probabilities

Default data (all customers, $t = 0.5$)

LDA (`balance + student`) catches 81 of 333 defaulters with 23 false alarms; logistic regression catches 105 with 40 false alarms – similar trade-offs, AUC around 0.95 for both.

Bayes classifiers with Python

Linear discriminant analysis

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = pd.get_dummies(df[["balance", "student"]], drop_first=True)
lda = LinearDiscriminantAnalysis().fit(X, df.default)
print(pd.crosstab(df.default, lda.predict(X)))    # 9644 23 / 252 81
```

Naive Bayes

```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB().fit(X, df.default)        # normal f_kj for every feature
print(pd.crosstab(df.default, nb.predict(X)))
```

▶ **Try it yourself:** notebook, section A.5 "Comparison" – open in Colab

Appendix summary – Bayes' theorem and Bayes classifiers

Total probability and Bayes' theorem

$$P(A) = \sum_j P(A \mid B_j)\, P(B_j), \qquad P(B_i \mid A) = \frac{P(A \mid B_i)\, P(B_i)}{P(A)}$$

In words: posterior $\propto$ likelihood $\times$ prior; the denominator comes from the law of total probability.

Classes and features: $P(Y = k \mid x) = \pi_k\, f_k(x) / \sum_\ell \pi_\ell\, f_\ell(x)$; the Bayes classifier picks the largest posterior.

- **Naive Bayes:** $f_k(x) = \prod_j f_{kj}(x_j)$ – independent features within each class.
- **LDA:** normal f_k with common variance; one variable: boundary $x^\star = \frac{\mu_0+\mu_1}{2} - \frac{\sigma^2}{\mu_1-\mu_0} \log \frac{\pi_1}{\pi_0}$.
- A rare class needs stronger evidence – the prior shifts the boundary.